

LOF TITAN — ESP32-S3 N16R8 Custom MicroPython Firmware + BLE Programming System

You are acting as the senior embedded-firmware engineer responsible for building a production-ready custom MicroPython firmware called **LOF TITAN**.

Do not stop after creating a plan or sample code. Work through the repository, toolchain setup, source implementation, firmware build, BLE protocol, persistent user-program management, test firmware, flashing scripts, documentation, and the companion web application.

Before modifying source, inspect the current MicroPython ESP32-S3 build system and the ESP-IDF version supported by the selected MicroPython release. Use a reproducible pinned toolchain rather than an unpinned moving branch.

The target is a custom PCB using:

MCU: ESP32-S3

Module configuration: ESP32-S3 N16R8

Flash: 16 MB

PSRAM: 8 MB Octal PSRAM

Firmware product name: LOF TITAN

MicroPython target: ESP32-S3 with Octal PSRAM support

Use the current stable MicroPython release unless a verified compatibility issue requires another pinned release.

1. PRIMARY PRODUCT GOAL

Build one firmware image that can be flashed to any LOF TITAN board.

The firmware must automatically determine the unique hardware identity of the board at runtime from its Bluetooth MAC address.

Do NOT create a different firmware binary for every board.

Every board flashed with the same firmware must automatically obtain a unique BLE name.

Example:

Bluetooth MAC:

A0:FE:12:34:56:78

Remove separators and take the **first four hexadecimal characters**:

A0FE

The BLE device name must become:

LOF_TITAN_A0FE

Another board with:

DC:54:75:11:22:33

must advertise as:

LOF_TITAN_DC54

The suffix must always be uppercase hexadecimal.

The naming format is strictly:

LOF_TITAN_XXXX

where `XXXX` is derived from the first two bytes / first four hexadecimal characters of that board's Bluetooth MAC address.

The BLE local name, GAP device name, device information returned to the web app, and system status screen must all use exactly the same generated device name.

Do not generate a random ID.

Do not derive the name from Wi-Fi SSID.

Do not store a manually assigned ID in the firmware.

Do not compile the MAC suffix into the firmware.

Read the real Bluetooth/interface MAC from the ESP32-S3 at runtime using the appropriate ESP-IDF MAC API supported by the installed IDF version.

Conceptually:

```
uint8_t mac[6];

read_bluetooth_mac(mac);

snprintf(
    device_name,
    sizeof(device_name),
```

```
"LOF_TITAN_%02X%02X",  
    mac[0],  
    mac[1]  
);
```

Verify the exact ESP-IDF API and enum for the pinned IDF version before implementation.

The generated name must remain the same after firmware reflash because it is based on the board's hardware MAC.

2. LOF TITAN PCB PIN MAP

Use the following PCB pin assignment as the official LOF TITAN hardware definition.

System / indicators

Hardware	GPIO
Red LED — LED1	GPIO47
Green LED — LED2	GPIO48
Buzzer	GPIO20
I2C SDA	GPIO7
I2C SCL	GPIO8

Sensor ports

Port	GPIO
S1	GPIO2
S2	GPIO1
S3	GPIO3
S4	GPIO4
S5	GPIO5

Ultrasonic port

Signal	GPIO
S1	GPIO6
S2	GPIO19

UART

Signal	GPIO
RX	GPIO18
TX	GPIO17

SPI

Signal	GPIO
SCK	GPIO35
MOSI	GPIO36
MISO	GPIO37
CS	GPIO38

Motor ports

Port	GPIO
M5	GPIO9, GPIO10
M6	GPIO11, GPIO12
M2	GPIO13, GPIO14
M4	GPIO15, GPIO16

Create one central board pin-definition file so these values are not duplicated throughout the project.

3. GPIO20 BUZZER REQUIREMENT

GPIO20 must remain available to normal MicroPython user programs.

The firmware is allowed to use GPIO20 for:

- startup tone
- no-program/error tone
- firmware confirmation sounds while the system is idle

However, GPIO20 must NOT be permanently reserved or blocked from MicroPython.

After a system tone completes:

1. stop PWM/tone generation;
2. disable/deinitialise the firmware buzzer peripheral;

3. leave GPIO20 in a safe state;
4. allow the user's program to initialise GPIO20 using `machine.Pin`, `machine.PWM`, etc.

For example, this must work from uploaded user code:

```
from machine import Pin, PWM
from time import sleep

buzzer = PWM(Pin(20))
buzzer.freq(1000)
buzzer.duty_u16(30000)

sleep(0.2)

buzzer.duty_u16(0)
buzzer.deinit()
```

Do not play connection/disconnection sounds while a user program is actively using GPIO20 unless there is a critical system reason.

4. BLE MUST ALWAYS BE AVAILABLE

This is a critical requirement.

BLE is a **firmware system service**, not a feature of the uploaded user's `main.py`.

BLE must initialise automatically whenever the board boots.

BLE must remain operational while:

- the board is idle;
- the 10-second startup timer is running;
- the user program is running;
- the user program contains loops;
- the device is waiting without a saved program;
- the web app disconnects;
- the system returns from STOP;
- a new program is uploaded.

The uploaded user program must NOT be responsible for starting the system BLE service.

Do not implement the critical BLE supervisor only inside a writable `/boot.py` or `/main.py`.

The BLE/supervisor implementation must be protected as part of the firmware itself, preferably native C/C++ integrated with the MicroPython ESP32 port and/or an immutable frozen module started from a protected native startup hook.

User filesystem deletion must not remove the BLE supervisor.

Whenever there is no active BLE connection:

```
BLE advertising = ON
```

Whenever a BLE client disconnects:

1. detect disconnect;
2. Green LED OFF;
3. Red LED ON;
4. restart advertising immediately;
5. keep or start the user program according to the state-machine rules below.

Do not deliberately shut down the BLE stack while user code is running.

5. BLE DEVICE NAME

At every boot:

```
Read Bluetooth MAC
    ↓
Convert first two bytes to uppercase hexadecimal
    ↓
Build device name
    ↓
LOF_TITAN_XXXX
    ↓
Configure GAP name
    ↓
Configure BLE advertisement
```

Example:

```
BLE MAC
A0:FE:12:34:56:78

Suffix
A0FE

BLE Name
LOF_TITAN_A0FE
```

Print the following to the serial diagnostic log:

```
LOF TITAN Firmware
BLE MAC       : A0:FE:12:34:56:78
Device ID     : A0FE
```

Device Name	: L0F_TITAN_A0FE
Flash	: 16 MB
PSRAM	: 8 MB
BLE	: READY

Also expose the full MAC and generated name through the BLE DEVICE INFO response.

6. LED BEHAVIOUR

GPIO47 = Red status LED.

GPIO48 = Green status LED.

BLE state controls these LEDs.

Disconnected / advertising:

RED	= ON
GREEN	= OFF

Connected:

RED	= OFF
GREEN	= ON

Programming:

Green may blink while data is being transferred.

Program successfully stored:

Green can provide a brief confirmation indication and then return to normal connected state.

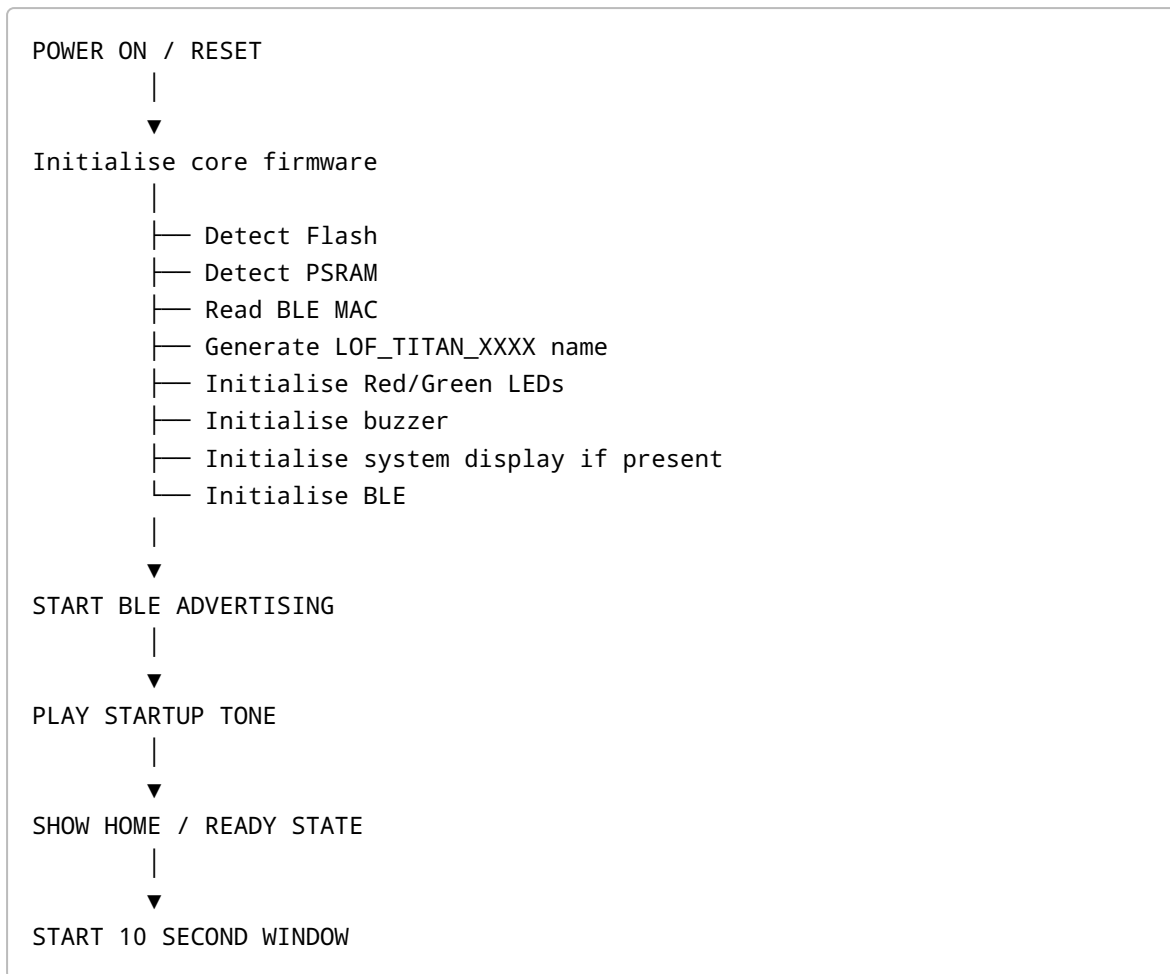
System error:

Use a clear Red LED blink pattern.

The BLE connection indication has priority.

7. REQUIRED BOOT WORKFLOW

Implement this exact logical startup sequence.



BLE must already be advertising during the 10-second window.

8. TEN-SECOND STARTUP WINDOW

When the firmware starts, wait up to 10 seconds before automatically starting the stored user program.

During this period BLE must remain fully available.

Display/log a countdown if useful.

Example:

```
LOF TITAN
LOF_TITAN_A0FE

BLE: WAITING
AUTO RUN: 10
```

Then:


```
10
9
8
...
1
```

9. CONNECTION DURING THE 10-SECOND WINDOW

If the web app connects before the 10-second timeout:

```
BLE CONNECTED
  ↓
RED OFF
GREEN ON
  ↓
Cancel automatic startup
  ↓
Enter APP CONTROL MODE
  ↓
Wait for commands
```

Do not automatically execute the stored program while the web app remains connected unless RUN is requested.

Supported commands must include:

STATUS

PROGRAM

RUN

STOP

RESET

Optionally prepare architecture for later commands such as:

FILES

DELETE

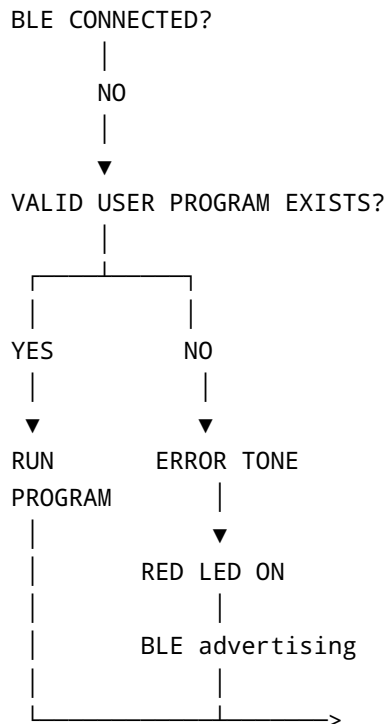
RENAME

REPL

but these are not required for the first stable version.

10. NO BLE CONNECTION AFTER 10 SECONDS

At the end of the 10-second startup window:



BLE advertising must remain active even while the user program runs.

11. USER PROGRAM STORAGE

Do not use the writable root `/main.py` as the firmware supervisor.

Store student/user code separately, for example:

```
/program/user.py
```

Use:

```
/program/user.tmp  
/program/user.py
```

```
/program/user.bak
/program/program.json
```

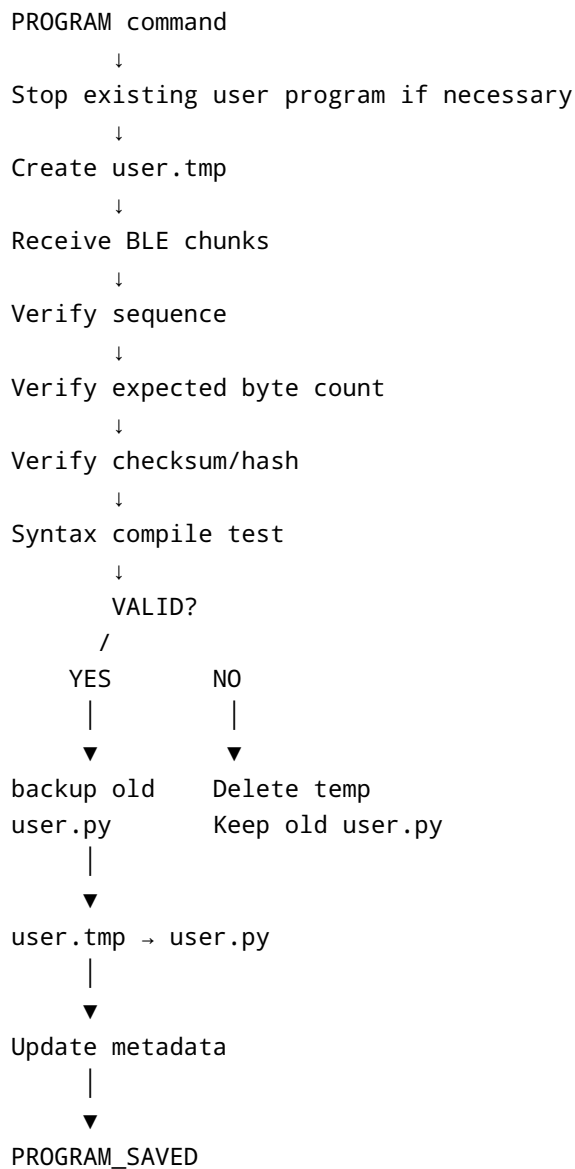
Critical firmware logic must remain outside the writable user-program area.

`user.py` must only run when requested by the firmware state machine.

12. ATOMIC PROGRAM UPLOAD

PROGRAM must never directly overwrite a known-good program during transfer.

Required workflow:



Power loss, BLE disconnection, malformed data, missing chunks, bad checksum, or syntax failure must not destroy the last known-good program.

13. BLE DISCONNECT DURING PROGRAMMING

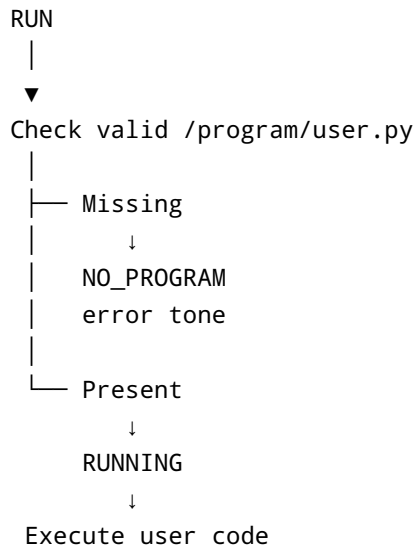
If BLE disconnects before PROGRAM completes:

1. stop accepting transfer;
2. mark transfer as incomplete;
3. delete `user.tmp`;
4. keep the previous `user.py`;
5. restart advertising immediately;
6. restore Red/Green BLE state;
7. if a valid previous program exists, follow the autonomous-run rule;
8. otherwise sound the no-program/error tone and wait for BLE.

Never promote a partially received file to `user.py`.

14. RUN COMMAND

When the app sends RUN:



The BLE subsystem must continue running.

The app must receive state:

`RUNNING`

If execution exits normally:

FINISHED

If it raises an exception:

ERROR

Return useful exception text to the app console when possible.

15. USER PROGRAM MUST NOT OWN THE SYSTEM SUPERVISOR

User Python code may access normal MicroPython APIs.

However, the user's code must not be able to accidentally remove the persistent firmware supervisor simply by:

```
import os
os.remove(...)
```

Critical system code must therefore be built/frozen into firmware.

The user filesystem is for:

- user.py
- user libraries
- program data
- future project assets

not for the essential BLE supervisor.

16. CONNECTING WHILE USER PROGRAM IS ALREADY RUNNING

This is an essential test.

Example:

```
User program running
      |
BLE advertising
      |
Web app connects
```



A BLE connection itself must not terminate an autonomous robot/program.

17. BLE DISCONNECT WHILE PROGRAM IS RUNNING

If BLE disconnects while user code is running:

```
BLE disconnect
  ↓
GREEN OFF
RED ON
  ↓
Restart advertising
  ↓
KEEP USER PROGRAM RUNNING
```

Do not stop the robot/project simply because Bluetooth disconnects.

18. BLE DISCONNECT WHILE SYSTEM IS IDLE

If the device was connected but no user program is currently running, then the client disconnects:

```
BLE disconnected
  ↓
RED ON
GREEN OFF
  ↓
Advertising ON
  ↓
Valid user.py?
  /
YES          NO
```



Use a short disconnect debounce if necessary to prevent accidental rapid state changes during transient reconnects.

19. STOP COMMAND

STOP must work even for typical long-running MicroPython programs such as:

```
while True:
    motor_run()
```

Implement STOP at the firmware/supervisor level.

Preferred behaviour:

```
STOP command
  ↓
Supervisor requests MicroPython interrupt
  ↓
Inject pending KeyboardInterrupt / safe VM interruption
  ↓
User program exits
  ↓
Clean user execution state
  ↓
Return system hardware to safe defaults where appropriate
  ↓
Display HOME / STOPPED
  ↓
BLE remains connected
```

Investigate and use the current MicroPython ESP32 mechanism for scheduling an interrupt into the VM rather than implementing an unsafe task kill.

If STOP cannot terminate code because the VM is stuck in a non-interruptible native operation, implement a documented recovery strategy.

Do not reboot the entire ESP32 for every normal STOP request.

A reboot may be used only as a last-resort recovery path.

20. PROGRAM COMMAND WHILE CODE IS RUNNING

If PROGRAM is selected while user code is running:

```
PROGRAM
↓
Request STOP
↓
Confirm user execution stopped
↓
Start upload
```

Do not write new code while an old Python execution is still modifying files or hardware.

21. RESET COMMAND

RESET should perform a controlled device restart.

After restart:

```
BLE starts automatically
↓
same MAC read again
↓
same LOF_TITAN_XXXX name
↓
startup tone
↓
10-second workflow begins again
```

22. SYSTEM STATE MACHINE

Implement explicit states rather than a collection of unrelated flags.

Suggested states:


```
BOOTING
WAITING_FOR_CONNECTION
CONNECTED_IDLE
PROGRAMMING
PROGRAM_READY
RUNNING
STOPPING
STOPPED
NO_PROGRAM
ERROR
```

BLE connection state may be maintained separately:

```
BLE_ADVERTISING
BLE_CONNECTED
```

Make transitions deterministic and log every important transition.

Example:

```
[STATE] BOOTING -> WAITING_FOR_CONNECTION
[BLE] advertising LOF_TITAN_A0FE
[BLE] connected
[STATE] WAITING_FOR_CONNECTION -> CONNECTED_IDLE
[CMD] RUN
[STATE] CONNECTED_IDLE -> RUNNING
```

23. GATT PROTOCOL

Create a custom LOF TITAN BLE GATT service.

At minimum implement characteristics conceptually equivalent to:

Characteristic	Direction	Purpose
CONTROL	App → TITAN	Commands
PROGRAM_DATA	App → TITAN	Python file chunks
STATUS	TITAN → App	State/results
CONSOLE	TITAN → App	Python/system output
DEVICE_INFO	TITAN → App	Device identity

Generate and document stable UUIDs.

Do not change the UUIDs between builds after the protocol is finalised.

24. CONTROL MESSAGE FORMAT

Use a small well-defined protocol.

JSON is acceptable for control messages.

Examples:

```
{"cmd": "STATUS"}
```

```
{"cmd": "RUN"}
```

```
{"cmd": "STOP"}
```

```
{"cmd": "RESET"}
```

PROGRAM initiation could be:

```
{  
  "cmd": "PROGRAM",  
  "filename": "user.py",  
  "size": 1248,  
  "checksum": "..."  
}
```

Responses:

```
{"status": "READY"}
```

```
{"status": "PROGRAMMING"}
```

```
{"status": "PROGRAM_SAVED"}
```

```
{"status": "RUNNING"}
```

```
{"status": "STOPPED"}
```

```
{"status": "NO_PROGRAM"}
```

```
{  
  "status": "ERROR",  
  "message": "SyntaxError on line 8"  
}
```

Keep messages versioned so the protocol can evolve.

25. PROGRAM DATA TRANSFER

BLE file transfer must be chunked.

Do not assume the entire Python file fits in one BLE write.

Implement:

```
START  
↓  
chunk 0  
chunk 1  
chunk 2  
...  
chunk N  
↓  
END  
↓  
checksum validation
```

Each transfer must have:

- transfer/session ID;
- sequence number;
- total size;
- received-byte count;
- integrity checksum;
- timeout handling;
- duplicate/missing chunk detection.

The web application should show upload percentage.

If larger BLE writes fail, reduce chunk size automatically rather than corrupting the upload.

26. DEVICE INFO

The app must be able to query something similar to:

```
{
  "product": "LOF TITAN",
  "device_name": "LOF_TITAN_A0FE",
  "device_id": "A0FE",
  "ble_mac": "A0:FE:12:34:56:78",
  "chip": "ESP32-S3",
  "flash_mb": 16,
  "psram_mb": 8,
  "firmware": "1.0.0",
  "micropython": "1.xx.x",
  "state": "CONNECTED_IDLE",
  "program_exists": true
}
```

Do not expose fabricated hardware information.

Read/detect values where practical.

27. MICROPYTHON BOARD DEFINITION

Create a proper MicroPython custom board configuration named:

LOF_TITAN

Target:

ESP32-S3

Memory:

16MB Flash + 8MB Octal PSRAM

Base the board configuration on the official ESP32-S3 Octal-SPIRAM setup but create a dedicated LOF TITAN board folder and configuration.

The board identification printed by MicroPython should clearly show LOF TITAN.

Example target identity:

MicroPython vX.Y.Z
LOF TITAN ESP32-S3 N16R8

Do not misidentify the board as generic ESP32.

28. FLASH PARTITIONING

Create an explicit partition table suitable for 16 MB flash.

Requirements:

- NVS/system storage;
- PHY/calibration partitions as required;
- sufficiently large MicroPython application partition;
- large persistent VFS/user partition.

There is no OTA requirement for firmware v1 unless implementation proves it useful.

Prefer a simple factory-app layout for v1.

Allow approximately 4 MB for application firmware and use most remaining space for persistent MicroPython/user storage.

Calculate actual boundaries with correct ESP32 alignment.

Do not blindly copy the partition table from classic 4 MB ESP32 firmware.

Document the final partition table in:

`docs/PARTITIONS.md`

and include:

Name
Type
Subtype
Offset
Size
Purpose

Verify the built MicroPython image fits with safe headroom.

29. FIRMWARE OUTPUTS

The completed build must provide the individual components and one merged image.

Place deliverables under:

```
dist/
```

Expected outputs:

```
dist/
├─ bootloader.bin
├─ partitions.bin
├─ micropython.bin
├─ LOF_TITAN_firmware.bin
├─ flash_args.txt
├─ checksums.txt
└─ VERSION.txt
```

`LOF_TITAN_firmware.bin` must be the convenient combined firmware suitable for flashing at the correct ESP32-S3 combined-image offset.

Do not guess component addresses.

Extract the actual offsets from the generated ESP-IDF/MicroPython build output and save them to `flash_args.txt`.

30. BUILD REPRODUCIBILITY

Create scripts:

```
scripts/setup.sh
scripts/build.sh
scripts/clean.sh
scripts/flash.sh
scripts/erase_flash.sh
```

If useful for Windows users, also create:

```
scripts/flash.ps1
scripts/erase_flash.ps1
```

Do not require someone to manually hunt through build directories for binaries.

`build.sh` must copy final artefacts to `dist/`.

Generate checksums for release binaries.

31. WEB PROGRAMMING APPLICATION

Create a companion application under:

`webapp/`

The first version can use Web Bluetooth.

The application must provide:

CONNECT
PROGRAM
RUN
STOP
RESET

and display:

- discovered TITAN name;
- connected/disconnected state;
- firmware information;
- Python code editor;
- upload progress;
- current execution state;
- console/errors.

The UI should identify boards by names such as:

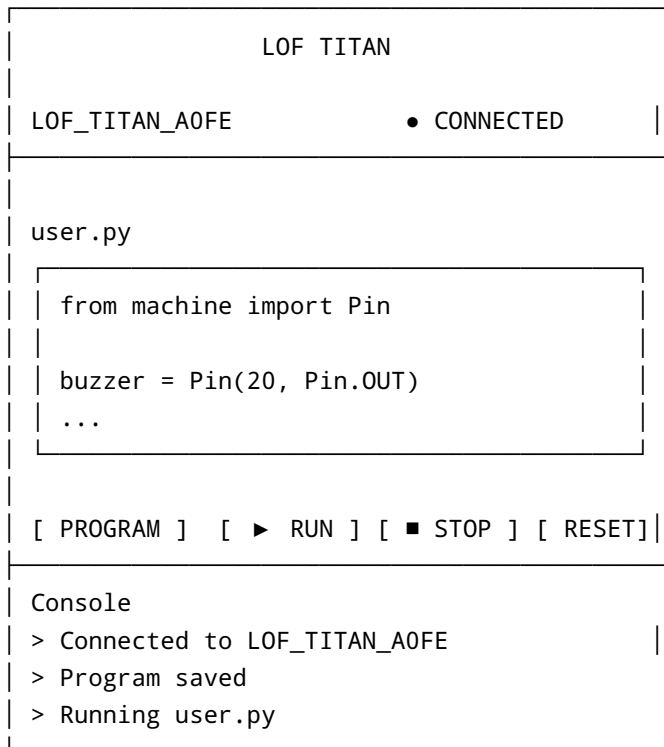
`LOF_TITAN_A0FE`

When scanning/selecting devices, prefer BLE devices whose advertised name begins with:

`LOF_TITAN_`

32. WEB APP MAIN SCREEN

Target a clean interface similar to:



Avoid unnecessary page scrolling on a normal desktop display.

33. PROGRAM BUTTON BEHAVIOUR

PROGRAM means:

Save/replace the program only.

It should not automatically run unless a separate option explicitly says:

PROGRAM & RUN

Required buttons:

PROGRAM
RUN
STOP
RESET

Optionally add:

PROGRAM & RUN

after the basic workflow is working.

34. BLE STATUS LED TEST

Create automated/manual test instructions for:

Disconnected

Expected:

```
Red GPIO47    = ON
Green GPIO48  = OFF

BLE advertising:
LOF_TITAN_XXXX
```

Connected

Expected:

```
Red GPIO47    = OFF
Green GPIO48  = ON
```

Disconnect

Expected:

```
Green OFF immediately
Red ON
Advertising restarts
```

35. REQUIRED MAC-NAME TESTS

This feature is mandatory.

Test 1:

Read actual Bluetooth MAC.

Example:

A0:FE:12:34:56:78

Expected:

LOF_TITAN_A0FE

Test 2:

Reflash exactly the same firmware on the same board.

Expected name:

LOF_TITAN_A0FE

It must not change.

Test 3:

Flash the same `LOF_TITAN_firmware.bin` to another physical board.

If its Bluetooth MAC begins:

7C:DF:...

Expected:

LOF_TITAN_7CDF

No manual editing or recompilation is allowed.

36. REQUIRED GPIO20 TEST

Upload:

```
from machine import Pin, PWM
from time import sleep

buzzer = PWM(Pin(20))

for freq in (500, 750, 1000, 1500):
    buzzer.freq(freq)
    buzzer.duty_u16(25000)
```

```
sleep(0.2)

buzzer.duty_u16(0)
buzzer.deinit()
```

Expected:

- program uploads;
- RUN executes it;
- GPIO20 produces the requested tones;
- firmware does not block GPIO20;
- BLE remains connected during execution.

37. REQUIRED AUTONOMOUS PROGRAM TEST

Program a simple persistent test:

```
from machine import Pin
from time import sleep

output = Pin(3, Pin.OUT)

while True:
    output.toggle()
    sleep(0.5)
```

Then:

1. disconnect BLE;
2. restart board;
3. do not connect for 10 seconds.

Expected:

```
startup
↓
BLE advertising
↓
10-second timeout
↓
stored program found
↓
program starts
↓
BLE continues advertising
```

38. REQUIRED CONNECT-WHILE-RUNNING TEST

While the autonomous test program is running:

1. open the web application;
2. connect to `LOF_TITAN_XXXX`.

Expected:

```
program continues running
BLE becomes connected
Green LED turns ON
Red LED turns OFF
STATUS reports RUNNING
```

The connection itself must not stop the program.

39. REQUIRED STOP TEST

Use:

```
while True:
    pass
```

or another continuous MicroPython loop.

Start it.

Then press:

`STOP`

Expected:

```
STOP received
↓
user code interrupted
↓
state becomes STOPPED
↓
BLE remains connected
↓
web application remains usable
```

This test is mandatory.

40. REQUIRED NO-PROGRAM TEST

Erase/remove the user program.

Restart board.

Do not connect during startup.

Expected after 10 seconds:

```
No valid user program
↓
Error beep
↓
Red LED remains ON
↓
BLE continues advertising
↓
LOF_TITAN_XXXX remains discoverable
```

41. REQUIRED FAILED-UPLOAD TEST

Start with a known-good `user.py`.

Begin a new PROGRAM transfer.

Disconnect Bluetooth halfway through.

Expected:

```
new upload rejected
user.tmp removed
old user.py preserved
BLE advertising restarts
```

Reconnect and RUN.

The previous known-good program must still execute.

42. REQUIRED BAD-SYNTAX TEST

Upload:

```
while True
    print("bad")
```

Expected:

```
syntax validation fails
new file not promoted
old valid program remains intact
app receives syntax error
```

43. LOGGING

Provide useful serial logs without excessive spam.

Example:

```
=====
LOF TITAN
Firmware 1.0.0
=====

Chip      : ESP32-S3
Flash     : 16 MB
PSRAM     : 8 MB
BLE MAC   : A0:FE:12:34:56:78
Device Name : LOF_TITAN_A0FE

[BLE] Initialising
[BLE] Advertising
[STATE] WAITING_FOR_CONNECTION
[AUTORUN] 10 seconds

[BLE] Connected
[STATE] CONNECTED_IDLE

[CMD] PROGRAM
[PROGRAM] Receiving 1824 bytes
[PROGRAM] Verification OK
[PROGRAM] Saved
```

```
[CMD] RUN
[STATE] RUNNING
```

44. REPOSITORY STRUCTURE

Organise the workspace cleanly, for example:

```
lof-titan/
|
|— firmware/
|   |— micropython/
|   |— board/
|   |— supervisor/
|   |— frozen/
|   └─ partitions/
|
|— webapp/
|
|— scripts/
|   |— setup.sh
|   |— build.sh
|   |— flash.sh
|   └─ erase_flash.sh
|
|— tests/
|
|— docs/
|   |— ARCHITECTURE.md
|   |— BLE_PROTOCOL.md
|   |— PINOUT.md
|   |— PARTITIONS.md
|   |— BUILD.md
|   |— FLASHING.md
|   └─ TESTING.md
|
|— dist/
|
└─ README.md
```

Adjust this structure if required by MicroPython conventions, but keep custom LOF code clearly separated from upstream code where practical.

45. DEVELOPMENT WORKFLOW

Work in phases.

Phase 1 — Environment

Set up:

- pinned MicroPython source;
- compatible pinned ESP-IDF;
- ESP32-S3 toolchain;
- mpy-cross;
- reproducible build commands.

Confirm a stock ESP32-S3 Octal-SPIRAM build succeeds before customisation.

Phase 2 — Custom LOF TITAN board

Create:

```
LOF_TITAN
```

Configure:

- ESP32-S3;
- 16 MB flash;
- 8 MB Octal PSRAM;
- custom board identity;
- partition table.

Build and flash a basic custom MicroPython firmware.

Confirm:

```
import gc
import machine
```

and verify PSRAM/flash behaviour.

Phase 3 — Device identity

Implement Bluetooth-MAC-based naming.

Confirm:

```
LOF_TITAN_XXXX
```


before implementing the rest of the BLE protocol.

Phase 4 — BLE supervisor

Implement:

- persistent BLE;
- advertising;
- connection callback;
- reconnection;
- LED state;
- STATUS command.

Phase 5 — Startup state machine

Implement:

- startup tone;
- 10-second window;
- connected mode;
- autonomous run;
- no-program error mode.

Phase 6 — Program management

Implement:

- temporary file;
- chunk transfer;
- checksum;
- syntax validation;
- backup;
- atomic replacement.

Phase 7 — Run/Stop

Implement safe Python execution and reliable STOP.

Test tight loops.

Phase 8 — Web application

Implement:

- BLE connection;
- editor;
- PROGRAM;
- RUN;
- STOP;

- RESET;
- console;
- progress.

Phase 9 — Hardware validation

Test on the actual LOF TITAN PCB.

Phase 10 — Release

Produce:

```
LOF_TITAN_firmware.bin
```

plus individual binaries, checksums, build instructions and flashing scripts.

46. IMPORTANT DESIGN RULES

Do not:

- build this as only a normal `main.py` ;
 - make BLE dependent on uploaded user code;
 - shut BLE off while user code runs;
 - hard-code one board's MAC address;
 - generate random BLE names;
 - require recompiling firmware for each device;
 - permanently reserve GPIO20;
 - overwrite a working program before a new upload is verified;
 - execute partially uploaded Python files;
 - stop a running autonomous program merely because BLE disconnected;
 - assume classic ESP32 flash offsets;
 - silently ignore build errors;
 - leave final binaries scattered inside build folders.
-

47. DOCUMENTATION REQUIREMENT

When implementation is complete, README must contain a quick-start section:

- ```
1. Build firmware
2. Connect LOF TITAN
3. Erase flash
4. Flash LOF_TITAN_firmware.bin
5. Restart
6. Listen for startup tone
```

7. Scan Bluetooth
8. Find LOF\_TITAN\_XXXX
9. Open LOF TITAN Web App
10. Connect
11. Program
12. Run
13. Stop

Also document how `XXXX` is derived from the Bluetooth MAC.

## 48. FINAL ACCEPTANCE CRITERIA

Do not mark the project complete until all of the following are demonstrated:

- ✓ ESP32-S3 target
- ✓ 16 MB flash detected/configured
- ✓ 8 MB Octal PSRAM detected/configured
- ✓ Custom LOF TITAN MicroPython build
- ✓ Same firmware binary works on multiple TITAN boards
- ✓ Actual Bluetooth MAC read automatically
- ✓ BLE name generated as LOF\_TITAN\_XXXX
- ✓ XXXX = first four hexadecimal characters of Bluetooth MAC
- ✓ Name remains the same after reflashing the same board
- ✓ BLE starts automatically
- ✓ BLE advertising while disconnected
- ✓ Red LED while disconnected
- ✓ Green LED while connected
- ✓ Startup tone works
- ✓ GPIO20 remains available from user MicroPython code
- ✓ 10-second startup window works
- ✓ Connection during countdown prevents automatic run
- ✓ Stored program runs autonomously when disconnected
- ✓ No-program condition produces error tone
- ✓ BLE remains available while program runs
- ✓ App can connect to an already-running program
- ✓ Connection does not stop the program
- ✓ BLE disconnect does not stop a running program
- ✓ STOP interrupts user execution
- ✓ PROGRAM safely replaces user code
- ✓ Interrupted upload preserves previous program
- ✓ Syntax-error upload preserves previous program
- ✓ RUN works
- ✓ STOP works
- ✓ RESET works
- ✓ STATUS works
- ✓ Web app can program/control board
- ✓ Individual binary components generated
- ✓ Combined LOF\_TITAN\_firmware.bin generated

- ✓ Build is reproducible
- ✓ Flashing scripts included
- ✓ Architecture and BLE protocol documented

---

## 49. FIRST ACTION FOR THE AGENT

Start by creating an implementation plan based on the actual MicroPython ESP32-S3 source tree.

Then immediately proceed with Phase 1.

Do not invent MicroPython or ESP-IDF APIs.

Inspect the exact source/version being used before integrating native functionality.

For every important implementation decision, prefer the existing MicroPython ESP32 architecture rather than creating parallel infrastructure unnecessarily.

After each phase:

1. build;
2. fix compile errors;
3. run relevant tests;
4. record what passed;
5. commit or checkpoint the working state;
6. continue to the next phase.

The goal is not merely to produce source files.

The final goal is a **flashable, tested LOF TITAN ESP32-S3 N16R8 MicroPython firmware and working BLE programming application**.